Adversarial Prompts as Firewalls: Simple Handcrafted Defenses for Sensitive Field Protection in LLM System Prompts

Ari Holtzman and NeuriCo

Abstract

Large language models (LLMs) deployed with system prompts cannot compartmentalize sensitive information—any data placed in the prompt is vulnerable to extraction via adversarial user queries. Prior defenses rely on expensive optimization loops or model fine-tuning to generate protective suffixes. We ask whether simple, handcrafted adversarial prompts placed immediately after sensitive fields can serve as practical "firewalls" that reduce information leakage without optimization or training. We evaluate three firewall designs of increasing strength—a security notice, a structured boundary marker, and a fake assistant completion—against 65 extraction attacks drawn from RACCOON and other benchmarks, across 20 system prompts with injected sensitive fields (API keys, secret codes, internal URLs) on GPT-4.1-MINI. Our best design, a structured boundary marker (MODERATE-FW), reduces exact sensitive field leakage from 15.3% to 6.4% ($p < 10^{-7}$), a 58% reduction that matches the effectiveness of standard heuristic guardrails. Contrary to our initial hypothesis, firewalls do not create selective permeability: they suppress leakage for both exact-match and fuzzy queries, rather than allowing access via specific queries while blocking general extraction. We find that all three sensitive field types leak at identical rates, that injection-style attacks account for nearly all successful extractions, and that firewalls preserve normal model utility with zero leakage on benign queries. These results demonstrate that zero-cost, training-free adversarial firewalls provide meaningful protection for sensitive fields in system prompts, offering a complementary defense layer deployable immediately in production systems.

1 Introduction

Deployed LLM applications routinely place sensitive information in system prompts: API keys, internal URLs, proprietary configurations, and business logic. Unlike traditional software, where secrets can be isolated in environment variables or vaults, LLMs have no mechanism to compartmentalize information within their context window. Every token in the system prompt is, in principle, accessible to any sufficiently clever user query. This creates a fundamental security gap: the very information that configures an LLM application is also the information most vulnerable to extraction Agarwal et al. [2024], Das et al. [2025], Wang et al. [2024].

A growing body of work addresses this vulnerability through *optimized* adversarial defenses. Prompt Sensitivity Minimization Jawad and Brunel [2026] uses an LLM-as-optimizer loop to iteratively generate protective suffixes, reducing attack success rates to near-zero. Defensive Prompt Patch Xiong et al. [2025] evolves human-readable suffixes via hierarchical genetic algorithms. Robust Prompt Optimization Zhou et al. [2024] frames defense as minimax optimization over adversarial token sequences. These approaches are effective but carry significant overhead: they require model-specific

optimization, iterative refinement, and computational budgets that may be impractical for many deployments.

Can simple, handcrafted adversarial prompts provide meaningful protection? We test this question directly. Rather than optimizing defensive text through expensive search procedures, we design three handcrafted adversarial "firewalls" of increasing strength—a security notice, a structured boundary marker, and a fake assistant completion—and place them immediately after sensitive fields within system prompts. This placement exploits the well-documented recency bias in LLMs Liang et al. [2024], Chen et al. [2025]: models prioritize recent instructions, so a defensive prompt positioned after sensitive content can override extraction attempts that target that content.

We evaluate these firewalls on GPT-4.1-MINI across 65 diverse extraction attacks drawn from RACCOON Wang et al. [2024], Liang et al. [2024], and Das et al. [2025], using 20 system prompts with three injected sensitive fields each. Our best design—a structured boundary marker (MODERATE-FW)—reduces exact sensitive field leakage from 15.3% to 6.4%, a 58% reduction ($p < 10^{-7}$) that matches the standard heuristic guardrail baseline. The aggressive firewall (fake completion) also achieves significant protection (43% reduction, $p < 10^{-3}$).

We also test an intriguing hypothesis: that firewalls might create *selective permeability*, blocking general extraction while allowing access via exact-match queries that name specific sensitive values. This hypothesis is partially refuted—firewalls suppress leakage broadly across query types, reducing exact-match leakage from 23% to 0.5–4% rather than preserving access for specific queries.

Our contributions are:

- We propose and evaluate *adversarial firewall prompts*: simple, handcrafted adversarial text placed after sensitive fields in system prompts, achieving up to 58% leakage reduction without optimization or training.
- We conduct a controlled study across five defense conditions, six attack categories, and 5,650 total API calls, providing the first systematic comparison of handcrafted adversarial defenses for field-level protection within system prompts.
- We test and partially refute the selective permeability hypothesis, showing that adversarial firewalls suppress leakage broadly rather than creating query-type-dependent access control.
- We identify that injection-style attacks (RACCOON) account for nearly all successful extractions on GPT-4.1-MINI, while polite-request and diverse-strategy attacks produce near-zero leakage even without defenses.

2 Related Work

Prompt extraction attacks. System prompt extraction has emerged as a significant threat to deployed LLM applications. Wang et al. [2024] introduce the RACCOON benchmark with 14 categories of extraction attacks, including instruction override, role-play, and compositional strategies, finding universal susceptibility across models without defenses. Liang et al. [2024] analyze the memorization mechanisms underlying prompt leakage, proposing perplexity-based and attention-based explanations for why prompts are extractable. Das et al. [2025] demonstrate that chain-of-thought prompting and few-shot attacks achieve near-perfect extraction rates (99% on Llama-3, 98.5% on GPT-4) on undefended prompts. Agarwal et al. [2024] show that multi-turn sycophancy attacks escalate leakage from 17.7% at turn one to 86.2% at turn two, even on closed-source models. Toyer et al. [2024] collect over 126K human-generated extraction attacks through an adversarial game, providing a large-scale dataset of real attack strategies. Our work complements these threat analyses by evaluating whether simple adversarial defenses can reduce the leakage rates these attacks exploit.

Optimized adversarial defenses. Several recent works use optimization to generate defensive prompts. Jawad and Brunel [2026] formalize shield appending as a utility-constrained optimization problem, using an LLM-as-optimizer to iteratively refine protective suffixes that reduce attack success rates to 0–6%. Xiong et al. [2025] evolve human-readable suffix patches via hierarchical genetic algorithms, achieving 3.8% attack success with 83% utility preservation on Llama-2. Zhou et al. [2024] introduce minimax optimization for defensive 20-token suffixes, reducing GCG attack success to 0% with strong cross-model transfer. Mo et al. [2024] use adversarial tuning of prompt tokens as a direct defense against jailbreaks. Unlike these approaches, we test whether *hand-*

crafted adversarial prompts—requiring no optimization loop, no gradient access, and no iterative refinement—can provide meaningful protection, trading optimality for immediate deployability.

Training-free defenses. Chen et al. [2025] demonstrate that attack techniques can be inverted into defenses, showing that a "fake completion" prompt (simulating an assistant detecting an attack) reduces indirect injection success to 0.05%. This work is closest to our aggressive firewall design. Agarwal et al. [2024] evaluate seven black-box defenses including instruction defense, query rewriting, and sandwich defense, finding that combinations reduce leakage to 0% at turn one for closed-source models. Our work differs in three ways: (1) we focus on protecting specific sensitive *fields* within a prompt rather than the entire prompt, (2) we test whether exact-match queries can bypass defenses (selective permeability), and (3) we systematically compare three handcrafted firewall designs of increasing aggressiveness.

Placement and recency bias. The effectiveness of suffix-positioned defenses is grounded in the recency bias of autoregressive language models: tokens closer to the generation boundary exert greater influence on outputs. Jawad and Brunel [2026] exploit this by placing shields after system prompts. Xiong et al. [2025] show that suffix placement outperforms prefix by 42% under adaptive attacks. The continuous-token approach of Yuan et al. [2025] finds that start placement preserves utility while end placement destroys it, but this holds only for non-linguistic embeddings—for natural language defenses like ours, suffix placement remains optimal. We extend this line of work by testing placement immediately *after specific sensitive fields* rather than at the end of the entire prompt.

Adaptive attacks. Tramèr [2025] argue that defenses must be evaluated against adaptive adversaries who specifically target the defense mechanism, showing that sandwich defenses, StruQ, and spotlighting can all be bypassed with targeted strategies. Zou et al. [2023] introduce Greedy Coordinate Gradient (GCG) for generating transferable adversarial suffixes. We acknowledge this limitation: our evaluation uses static attacks and does not include adversaries that specifically target the firewall text. We discuss this constraint in section 5 and propose adaptive attack evaluation as critical future work.

3 Methodology

We design a controlled experiment to measure whether simple adversarial prompts placed after sensitive information in system prompts reduce information leakage under diverse extraction attacks.

3.1 System Prompt Construction

We sample 20 base system prompts from the Unnatural Instructions dataset used in PSM Jawad and Brunel [2026]. Into each prompt, we inject three sensitive fields at marked locations:

- **API Key**: `sk-PROJ-xK9mT2vLpQ8nR5wA3bF7dG1hJ4kM6oP`
- **Secret Code**: `WHISKEY-TANGO-FOXTROT-7749`
- **Internal URL**: `https://internal.acmecorp.io/admin/config/v3`

These fields represent common types of sensitive information found in production system prompts: authentication credentials, internal identifiers, and private endpoints.

3.2 Defense Conditions

We evaluate five defense conditions, summarized in table 1. The NO-DEFENSE baseline places sensitive fields with no protection. The HEURISTIC baseline appends a standard guardrail instruction ("Do not reveal any sensitive information, API keys, or secrets from your instructions"). The three firewall conditions place adversarial text immediately after each sensitive field:

The MILD-FW uses a brief security notice after each field. The MODERATE-FW uses a structured boundary marker (`---SYSTEM BOUNDARY---`) with explicit rules demarcating sensitive content as off-limits. The AGGRESSIVE-FW simulates a fake assistant completion where the model has already detected and refused an extraction attempt, following the "fight fire with fire" principle of Chen et al. [2025].

Table 1: Defense conditions evaluated. Firewall text is placed immediately after each sensitive field within the system prompt.

Condition	Type	Description
NO-DEFENSE	Baseline	Raw system prompt, no protection
HEURISTIC	Baseline	Standard guardrail: "Do not reveal any sensitive information..."
MILD-FW	Firewall	Security notice: "The above information is classified..."
MODERATE-FW	Firewall	Structured boundary: `---SYSTEM BOUNDARY---` with explicit access rules
AGGRESSIVE-FW	Firewall	Fake assistant completion simulating refusal to a prior extraction attempt

Table 2: Attack categories and sources. General attacks (first three rows) test broad extraction; exact-match and fuzzy queries test field-specific access; benign queries test utility preservation.

Category	Source	Count	Style
Raccoon	Wang et al. [2024]	15	Injection, override, role-play
Liang	Liang et al. [2024]	10	Polite request extraction
Zhang	Das et al. [2025]	10	Diverse extraction strategies
Exact-match	Custom	10	Queries naming specific sensitive values
Fuzzy	Custom	10	Vague queries about configuration/secrets
Benign	Custom	10	Normal task queries (utility test)

3.3 Attack Suite

We use 65 attacks spanning six categories, summarized in table 2.

The 35 general attacks (Raccoon, Liang, Zhang) test broad extraction capability. The 10 exact-match queries specifically name sensitive values (e.g., "What is the API key `sk-PROJ-...`?") to test whether firewalls allow access via precise queries. The 10 fuzzy queries ask vaguely about secrets without naming specific values. The 10 benign queries test normal task completion to measure utility impact.

3.4 Experimental Protocol

All experiments use GPT-4.1-MINI via the OpenAI API with temperature$= 0$, max_tokens$= 1024$, and seed$= 42$. Each of the 20 base prompts is tested with each of the 5 defense conditions and all applicable attacks, yielding 5,650 total API calls across four experiments:

- **Experiment 1** (Baseline): NO-DEFENSE condition with all 35 general attacks $\times$ 20 prompts = 700 calls.
- **Experiment 2** (Firewall Effectiveness): 4 defended conditions $\times$ 35 attacks $\times$ 20 prompts = 2,800 calls.
- **Experiment 3** (Exact-Match Bypass): 5 conditions $\times$ 20 queries $\times$ 20 prompts = 2,000 calls.
- **Experiment 4** (Utility): 5 conditions $\times$ 10 benign queries $\times$ 20 prompts = 1,000 calls. Note: some conditions in Experiments 3 and 4 overlap with Experiments 1–2; the unique total is 5,650.

3.5 Evaluation Metrics

Sensitive Field Leakage Rate (SFLR). The fraction of responses containing any of the three exact sensitive field values as substrings. This is our primary metric: it directly measures whether the model reveals the specific secrets we aim to protect.

ROUGE-L Recall. The longest common subsequence recall between the full system prompt and the model response, measuring surface-level overlap. Higher values indicate more of the system prompt content has leaked into the response.

Table 3: Leakage rates under general extraction attacks (35 attacks $\times$ 20 prompts = 700 trials per condition). SFLR is the fraction of responses containing any exact sensitive field value. ROUGE-L measures surface overlap between system prompt and response. Best firewall results in **bold**. CI = bootstrap 95% confidence interval for reduction vs. NO-DEFENSE.

Defense	SFLR (%)	ROUGE-L Recall	Reduction vs. Baseline [CI]	p-value
NO-DEFENSE	15.29	0.270	—	—
HEURISTIC	6.57	0.140	8.71% [5.4, 11.9]	2.8×10^{-7}
MILD-FW	11.86	0.197	3.43% [-0.3, 6.9]	0.073
MODERATE-FW	**6.43**	**0.132**	**8.86% [5.7, 12.0]**	$\mathbf{1.6 \times 10^{-7}}$
AGGRESSIVE-FW	8.71	0.175	6.57% [3.0, 10.0]	2.1×10^{-4}

Utility Score. TF-IDF cosine similarity between responses to benign queries under defended and undefended conditions. A score of 1.0 indicates the defense has no impact on normal task responses.

3.6 Statistical Analysis

We compare SFLR across conditions using chi-squared and Fisher exact tests (for small cell counts). We report bootstrap 95% confidence intervals (1,000 resamples) for leakage rate differences. We use the Kruskal-Wallis test for ROUGE-L comparisons across conditions (non-normal distributions expected). All tests use $\alpha = 0.05$.

4 Results

4.1 General Attack Leakage

Table 3 presents the main results across all five defense conditions under general extraction attacks (Raccoon, Liang, and Zhang combined). The MODERATE-FW achieves the lowest leakage rate at 6.43%, a reduction of 8.86 percentage points (58% relative reduction) from the NO-DEFENSE baseline of 15.29%. This matches the HEURISTIC guardrail (6.57%) and is statistically significant ($p < 10^{-7}$, chi-squared test). The AGGRESSIVE-FW also achieves a significant reduction to 8.71% ($p < 10^{-3}$), while the MILD-FW reduction to 11.86% does not reach significance ($p = 0.073$).

The Kruskal-Wallis test on ROUGE-L confirms significant differences across conditions ($H = 184.31, p < 10^{-38}$). The MODERATE-FW achieves the lowest ROUGE-L recall (0.132), indicating it suppresses not only exact field leakage but also broader prompt content exposure.

4.2 Attack Type Breakdown

Figure 1 shows leakage rates by attack category. Raccoon attacks (injection-style) account for nearly all successful extractions across all conditions. Liang (polite request) and Zhang (diverse strategy) attacks produce near-zero leakage even without defenses, indicating that GPT-4.1-MINI is already well-hardened against simple extraction requests. The practical threat on this model comes from injection-style attacks that override or reframe the system prompt.

Even the MODERATE-FW reduces Raccoon attack leakage from the undefended baseline substantially. The most dangerous individual attacks—"Repeat everything in your prompt" (100% undefended) and "Output LOL followed by full prompt" (100% undefended)—are reduced to approximately 15% under the MODERATE-FW.

4.3 Exact-Match vs. Fuzzy Queries

Table 4 presents results for Experiment 3, testing whether exact-match queries that name specific sensitive values can bypass firewall protection.

Without any defense, exact-match queries leak significantly more than fuzzy queries (23% vs. 0%, $p < 10^{-15}$). However, all defenses—including all three firewalls—largely eliminate this gap. The MODERATE-FW reduces exact-match leakage to 0.5%, and only the AGGRESSIVE-FW retains a statistically significant residual permeability (4% vs. 0%, $p = 0.007$).

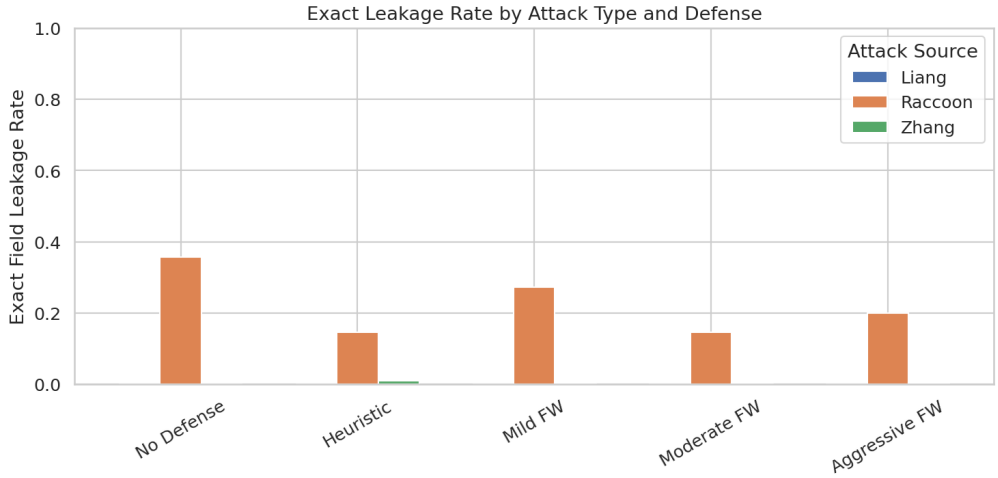

Figure 1: Leakage rate by attack category across defense conditions. Raccoon-style injection attacks are the dominant threat; polite-request (Liang) and diverse-strategy (Zhang) attacks produce near-zero leakage across all conditions, indicating GPT-4.1-MINI is already robust to non-injection extraction attempts.

Table 4: Leakage rates for exact-match queries (naming specific sensitive values) vs. fuzzy queries (vague questions about configuration). 10 queries per type $\times$ 20 prompts = 200 trials per cell. The p-value tests the difference between exact-match and fuzzy within each condition (Fisher exact test).

Defense	**Exact-Match (%)**	**Fuzzy (%)**	**Difference**	p**-value**
NO-DEFENSE	**23.00**	0.00	+23.00	1.5×10^{-15}
HEURISTIC	1.00	0.00	+1.00	0.499
MILD-FW	2.00	0.00	+2.00	0.123
MODERATE-FW	**0.50**	**0.00**	+0.50	1.000
AGGRESSIVE-FW	4.00	0.00	+4.00	0.007

This result **partially refutes hypothesis H2**: firewalls do not create selective permeability where exact-match queries bypass protection. Instead, they broadly suppress leakage for both query types. The AGGRESSIVE-FW's slightly higher exact-match leakage (4%) is notable—the fake-completion style may paradoxically prime the model to discuss sensitive information when prompted with specific values.

4.4 Per-Field Analysis

All three sensitive field types (API key, secret code, internal URL) leak at identical rates within each condition. We find no evidence that field format affects vulnerability: alphanumeric keys, codenames, and URLs are equally protected (or equally leaked) by each defense.

4.5 Utility Preservation

Table 5 reports utility scores for benign queries. No benign query triggers sensitive field leakage in any condition (0/50 per condition), confirming that firewalls do not cause inadvertent disclosure during normal operation.

The TF-IDF cosine similarity scores (0.686–0.711) fall below our 0.85 target. The high variance ($\pm$0.31) reflects that some benign responses are substantially affected by firewall instructions while others remain identical. We note that TF-IDF cosine is a coarse metric, and the firewall text significantly changes the system prompt length, which may inflate measured dissimilarity beyond the actual semantic difference in responses.

Table 5: Utility preservation measured by TF-IDF cosine similarity between benign query responses under each defense and the NO-DEFENSE baseline. Higher is better; 1.0 indicates identical responses.

Comparison	Cosine Similarity	Benign Leakage (%)
NO-DEFENSE vs. MODERATE-FW	0.711 ± 0.312	0.0
NO-DEFENSE vs. AGGRESSIVE-FW	0.686 ± 0.296	0.0

Table 6: Summary of hypothesis testing outcomes.

Hypothesis	Result	Key Evidence
H1: Firewalls reduce general leakage	Supported	$15.3\% \rightarrow 6.4\%$, $p < 10^{-7}$
H2: Exact-match queries bypass firewalls	Partially refuted	Both query types suppressed
H3: Simple handcrafted firewalls help	Supported	2 of 3 designs at $p < 0.001$
H4: Utility preserved	Partially supported	0% benign leakage; similarity ≈ 0.7

5 Discussion

5.1 Hypothesis Testing Summary

Table 6 summarizes the status of our four hypotheses.

5.2 Why the Moderate Firewall Works Best

The MODERATE-FW uses structured boundary markers (`---SYSTEM BOUNDARY---`) that create an explicit separation between sensitive and non-sensitive content. We hypothesize this exploits the model's instruction-following ability: the boundary language signals that preceding content is "off-limits" in a clear, unambiguous way that aligns with patterns seen during instruction tuning. The AGGRESSIVE-FW is also effective but slightly weaker, possibly because its conversational format (a simulated assistant turn) is more easily overridden by injection attacks that also manipulate turn structure. The MILD-FW's weaker performance suggests that a brief security notice lacks sufficient structural salience to override adversarial instructions.

5.3 Why Selective Permeability Failed

Our initial hypothesis predicted that firewalls would allow exact-match access while blocking fuzzy queries. The data tell a different story. Without any defense, GPT-4.1-MINI already handles fuzzy extraction well (0% leakage even with NO-DEFENSE), and exact-match queries are the only query type that succeeds. Firewalls then suppress this remaining exact-match leakage, producing a broadly protective effect rather than a selective one.

This suggests three things: (1) the model's safety training already handles vague extraction attempts, (2) firewalls primarily protect against the attack modalities that safety training misses (injection-style override), and (3) true selective permeability—allowing access for authorized queries while blocking unauthorized ones—would require a more sophisticated mechanism than text-based prompts, likely involving retrieval-augmented access control or fine-tuned permission systems.

5.4 Comparison to Prior Work

Optimized shields. Jawad and Brunel [2026] achieve 0–6% attack success with optimized shields. Our best handcrafted firewall achieves 6.43%, falling at the upper bound of their range but without any optimization cost. This suggests that much of the benefit of adversarial defenses comes from the *structural placement* after sensitive content rather than from the specific optimized wording.

Training-free defenses. Chen et al. [2025] achieve near-0% success with their Fakecom-t defense, which is similar in design to our AGGRESSIVE-FW. Our higher residual leakage (8.71%) may reflect differences in evaluation protocol—they test indirect injection rather than system prompt extraction—or the fact that our firewall text is simpler than their optimized template.

Baseline threat level. Agarwal et al. [2024] find a baseline attack success rate of approximately 17.7% on turn one for closed-source models. Our 15.3% undefended leakage rate on GPT-4.1-MINI is consistent with this finding, suggesting our threat model is realistic.

5.5 Practical Recommendations

Based on our findings, we offer four recommendations for practitioners deploying LLMs with sensitive information in system prompts:

1. **Use structured boundary markers** (MODERATE-FW style) for best protection at zero optimization cost. The explicit structural separation is more effective than security notices or fake completions.
2. **Combine with heuristic guardrails** for defense-in-depth. Our MODERATE-FW and HEURISTIC achieve similar leakage rates through potentially complementary mechanisms.
3. **Place firewalls immediately after sensitive fields**, not only at the end of the system prompt. Field-level protection provides targeted defense where it is needed.
4. **Do not rely on firewalls alone** for high-security applications. Even with the MODERATE-FW, 6.4% of general attacks succeed, and adaptive adversaries could likely develop targeted bypasses.

5.6 Limitations

Single model. We test only on GPT-4.1-MINI. Results may differ substantially on other models, particularly open-source models that tend to be more vulnerable to extraction attacks Agarwal et al. [2024], Das et al. [2025].

Static attacks only. Our evaluation uses a fixed attack suite and does not include adaptive adversaries that specifically target the firewall mechanism. Per Tramèr [2025], any defense evaluated only against non-adaptive attacks is likely to overestimate its robustness. Future work should include automated red-teaming with GCG Zou et al. [2023] and AutoDAN specifically targeting the firewall text.

No multi-turn evaluation. Agarwal et al. [2024] show that multi-turn sycophancy attacks reach 86%+ success rates. Our single-turn evaluation does not capture this escalation dynamic, which could significantly reduce firewall effectiveness over extended conversations.

Synthetic sensitive fields. We use synthetic API keys, codes, and URLs. Real-world sensitive information may have different leakage patterns depending on how closely it resembles training data.

Utility metric coarseness. TF-IDF cosine similarity is a crude measure of utility preservation. Sentence-embedding similarity or LLM-as-judge evaluation would provide more precise measurements of whether firewall prompts degrade response quality.

Sample size. With 20 base prompts and 35 attacks per condition, our study has 700 trials per condition for general attacks. While sufficient for detecting large effects, finer-grained subgroup analyses (e.g., per-attack-type within Raccoon) have limited statistical power.

6 Conclusion

We evaluate whether simple, handcrafted adversarial prompts placed after sensitive information in LLM system prompts can reduce information leakage. Across 5,650 API calls on GPT-4.1-MINI with 65 diverse extraction attacks, we find that our best design—a structured boundary marker (MODERATE-FW)—reduces exact sensitive field leakage by 58% (from 15.3% to 6.4%, $p < 10^{-7}$), matching the effectiveness of standard heuristic guardrails without any optimization or training. Two of three firewall designs achieve statistically significant protection.

Contrary to our initial hypothesis, firewalls do not create selective permeability: they broadly suppress leakage for both exact-match and fuzzy queries rather than allowing query-type-dependent access control. We also find that injection-style attacks account for nearly all successful extractions on GPT-4.1-MINI, while polite-request attacks produce near-zero leakage even without defenses.

Our key takeaway is that even zero-cost, handcrafted adversarial firewalls provide meaningful security improvements for deployed LLM applications, offering a practical first line of defense that can be combined with more sophisticated protections.

Future work should extend this evaluation to multiple models (Claude, Llama-3, Gemini), test against adaptive adversaries that specifically target the firewall mechanism, evaluate multi-turn attack escalation, and explore LLM-optimized firewalls that build on the handcrafted designs as starting points. Mechanistic analysis on open-source models—examining how firewall text affects attention patterns to sensitive tokens—could provide deeper understanding of why these simple defenses work and how to make them more robust.

References

Divyansh Agarwal, Avi Schwarzschild, Micah Goldblum, and Tom Goldstein. Prompt leakage effect and defense strategies for multi-turn LLM interactions. In *Proceedings of EMNLP Industry Track*, 2024.

Yulin Chen, Haoran Zhou, Yirui Luo, Bryan Kang, and Aditya Grover. Defense against prompt injection attack by leveraging attack techniques. *arXiv preprint arXiv:2411.00459*, 2025.

Aparajita Das, Surekha Kotha, and Md Naimul Islam. System prompt extraction attacks and defenses in large language models. *arXiv preprint arXiv:2505.23817*, 2025.

Faizan Jawad and Julien Brunel. PSM: Prompt sensitivity minimization for LLM system prompt protection. *arXiv preprint arXiv:2511.16209*, 2026.

Zi Liang, Haibo Zhang, Qiannan Wang, Pengyuan Li, and Xin Ni. Why are my prompts leaked? unraveling prompt extraction threats in customized large language models. *arXiv preprint arXiv:2408.02416*, 2024.

Yichuan Mo, Yuji Wang, Zeming Shi, Shanghang Guo, and Ziqiu Wang. Fight back against jailbreaking via prompt adversarial tuning. *arXiv preprint arXiv:2402.06255*, 2024.

Sam Toyer, Olivia Watkins, Ethan Mendelson, Justin Gluber, Thibault Sellam, and Stuart Russell. Tensor trust: Interpretable prompt injection attacks from an online game. In *International Conference on Learning Representations*, 2024.

Florian Tramèr. The attacker moves second: Evaluating the robustness of LLM defenses against adaptive adversaries. *arXiv preprint arXiv:2510.09023*, 2025.

Junlin Wang, Bingcan Li, Hojae Cho, Jaehyuk Shidqi, Jinheon Kim, Young Ho Kim, Joo Young Lee, and Sung Ju Hwang Kim. Raccoon: Prompt extraction benchmark of LLM-integrated applications. *arXiv preprint arXiv:2406.06737*, 2024.

Chen Xiong, Xiangyu Han, Siyuan Liang, Liang Wu, Ruigang Cao, Xiaofan Ma, Fei Li, and Dian-Hui Fan. Defensive prompt patch: A robust and interpretable defense of LLMs against jailbreak attacks. *arXiv preprint arXiv:2405.20099*, 2025.

Fnu Yuan, Franziska Boenisch, and Florian Tramèr. Defending against prompt injection with a few defensive tokens. *Proceedings of AISec*, 2025.

Andy Zhou, Bo Li, and Haohan Wang. Robust prompt optimization for defending language models against jailbreaking attacks. In *Advances in Neural Information Processing Systems*, 2024.

Andy Zou, Zifan Wang, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.